Relazione stage interno Luca Santarelli

1. Obiettivo del progetto

Durante il mio stage, mi sono dedicato all'implementazione di un progetto che ha richiesto l'integrazione e l'uso di diversi componenti hardware e software, tra cui un misuratore di energia UPM209, un microcontrollore ESP32, un modulo di comunicazione Atom DTU per RS485 e il protocollo di rete Modbus. L'obiettivo era quello di creare un software in grado di monitorare in tempo reale i dati energetici dell'UPM209 e salvarli nella flash dell'ESP32s3 in formato JSON.

2. Componenti

2.1 Esp32s3

L'esp32s3 è un microcontrollore dual-core prodotto da Espressif Systems in grado di funzionare ad una frequenza fino a 240 MHz utile per la gestione di protocolli di comunicazione. E' dotato di una memoria con 512 KB di RAM interna e 8MB di memoria flash esterna. E' inoltre fornito di Bluetooth 5.0 e Wi-Fi che consentono la comunicazione wireless per le applicazioni IoT.

Dispone di GPIO con 45 pin programmabili che permette di collegare numerosi sensori e dispositivi esterni (questo da modificare).

E' supportato da un maturo ambiente di sviluppo denominato ESP-IDF che permette agli sviluppatori di sviluppare o migrare applicazioni con facilità nella paittaforma ESP32-S3 e continuare a lavorare con gli strumenti ESP-IDF e le API.

2.2 UPM209

L'UPM209 è un analizzatore di rete progettato dall'azienda Algodue Elettronica ed offre il monitoraggio completo dei parametri elettrici in sistemi trifase e monofase. Permette di misurare un gran numero di variabili in tempo reale tra cui: Correnti di fase, potenze attiva, reattiva e apparente , frequenza di rete ed altri.

Ha un interfaccia seriale RS485 che implementa il protocollo MODBUS RTU consentendo la comunicazione e lo scambio di dati con i computer. E' dotato di un display LED integrato che permette di visualizzare più parametri allo stesso tempo.

2.3 MODBUS

Il Modbus è un protocollo di comunicazione seriale utilizzato in ambito industriale per lo scambio di dati tra dispositivi come sensori, attuatori e controller. Il protocollo Modbus RTU permette al dispositivo centrale che rappresenta il Master di ottenere informazioni e scrivere registri nei dispositivi collegati ovvero i dispositivi slave.

Comunicazione in Modbus RTU

La comunicazione avviene inviando blocchi di dati denominati frame. Ogni frame è composto dalle seguenti parti:

- Indirizzo del dispositivo slave: un identificativo univoco del dispositivo destinatario.
- Codice funzione: indica quale operazione deve essere eseguita. I codici sono:
  0x03 per la lettura dei registri holding, ossia registri che contengono parametri configurabili o dati statici.
  0x04 per la lettura dei registri input, che forniscono informazioni in tempo reale ma non configurabili.
- Dati trasmessi: i valori dei registri richiesti o da impostare, espressi in formato binario.
- CRC (Cyclic Redundancy Check): un checksum utilizzato per verificare l'integrità dei dati trasmessi.

Tutti gli Slave ascoltano i dati trasmessi dal Master sul bus ma solo lo Slave indirizzato risponde alla richiesta, mentre gli altri la ignorano. Dopo l'interrogazione lo Slave selezionato invia una risposta al Master con la stessa struttura del frame ricevuto. Il Master può successivamente avviare una nuova interrogazione con lo stesso dispositivo o con un altro Slave procedendo allo stesso modo.

Lo Slave deve rispondere rapidamente per non rallentare la comunicazione con gli altri dispositivi. Per evitare blocchi, il Master gestisce un tmpo massimo di attesa (Timeout). Se lo Slave non risponde entro questo limite, la comunicazione continua con altre operazioni.

Ogni byte del frame viene trasmesso sul bus seriale RS485 utilizzando 11 bit: un bit di start (0) per il sincronismo, 8 bit di dati inviati dal meno significativo al più significativo, un bit di parità per la rilevazione di errori e un bit di stop (1) per terminare la trasmissione del byte.

Registri Modbus

I registri Modbus sono l'elemento centrale del protocollo. Ogni registro è largo 16 bit (1 word) e può essere combinato con altri registri per rappresentare valori più complessi:

- Valori a 16 bit (1 word): per dati semplici come numeri interi.
- Valori a 32 bit (2 word): per valori più grandi o con rappresentazione in virgola mobile (IEEE 754).
- Valori a 64 bit (4 word): per valori molto estesi.

2.4 Rs485 e Modbus RTU

L'RS-485 è un'interfaccia di comunicazione seriale progettata per trasferire dati su lunghe distanze attraverso cavi a doppino intrecciato.

Consente la connessione di più dispositivi sulla stessa linea (punto-multipunto) e fa uso di una trasmissione differenziale per ridurre le interferenze elettromagnetiche.

Il flusso di dati è semiduplex, permettendo di inviare e ricevere alternativamente. E' spesso utilizzato insieme al Modbus RTU e consente lo scambio di informazioni tra un dispositivo master e diversi dispositivi slave, con meccanismi per evitare conflitti e garantire l'integrità dei dati.

2.5 Atom DTU

L'atom DTU è un'unita di trasmissione dati programmabile che integra le funzioni di comunicazione NB-IoT.

Equipaggia un modulo di comunicazione SIM7028 che permette le comunicazioni wireless a basso consumo energetico e con un'ampia copertura.

Inoltre l'atom DTU è costituito da porte seriali e GPIO per collegare vari sensori e attuatori, permettendo di raccogliere dati e controllare dispositivi a distanza. E' ideale per scenari a bassa latenza e basso throughput come il monitoraggio remoto, il tracciamento di asset, il controllo remoto e la telemedicina.

3. Implementazione del codice

Studio iniziale

All'inizio ho innanzitutto approfondito le specifiche tecniche dell'UPM209 attraverso il manuale messo a disposizione nel sito di Algodue Elettronica, i creatori del dispositivo. In particolare per identificare l'indirizzo del dispositivo slave, la velocità di comunicazione, la parità i registri associati a ciascun valore ed il loro formato numerico.
Dopodiché ho studiato come avviene la comunicazione RS485 nel protocollo Modbus al fine di ottenere i dati dall'analizzatore elettrico e in fine il funzionamento e le specifiche dell'Atom DTU tra cui i pin necessari da utilizzare per la trasmissizione e la ricezione dei dati.

Sviluppo del software

Il software principale è stato scritto utilizzando il framework ESP-IDF nell'ambiente di sviluppo Visual Studio Code e la libreria ESP-Modbus utilizzando le API messe a disposizione per il modbus master. Per il salvataggio dei dati in formati json nella flash sono state utilizzate la libreria Cjson per il parsing dei dati in formato JSON e il file system SPIFFS per il salvataggio dei file JSON nella flash del microcontrollore esp32s3

Configurazione iniziale

Nella configurazione iniziale ho definito la porta seriale da utilizzare (UART1). La modalità RTU, i parametri della connessione, tra cui baud rate (19200 bps) e configurazione della parità (nessuna parità), l'indirizzo dello slave modbus, i pin utilizzati dall'ESP32s3 per la trasmissione (TX), la ricezione (RX) e il controllo RTS (Request to Send)

Gestione dei registri Modbus dell'UPM209:
Ho analizzato i registri disponibili per il misuratore UPM209 e li ho organizzati in una struttura di registri personalizzata, mappando ciascun parametro ai rispettivi indirizzi Modbus e definendone la dimensione e il tipo di dato

Inizializzazione spiffs

Attraverso la funzione init_spiffs() ho configurato il file system SPIFFS (SPI Flash File System). Questo consente di salvare i dati letti dai registri dell'UPM209 in un formato JSON.
L'inizializzazione include il montaggio della partizione SPIFFS, configurata per accogliere fino a 5 file, con un'opzione per il formato automatico in caso di errore. Dopo il montaggio del file system, viene visualizzata sulla console la dimensione totale e quella utilizzata della partizione SPIFFS.

Inizializzazione modbus

Attraverso la funzione master_init() ho inizializzato il Modbus Master, definendo:

- La porta seriale da utilizzare (UART1).
- La modalità RTU.
- I parametri della connessione, tra cui baud rate (19200 bps) e configurazione della parità (nessuna parità).

Vengono inoltre assegnati i pin utilizzati dall'ESP32 per la trasmissione (TX), la ricezione (RX) e il controllo RTS (Request to Send).

Lettura dei registri Modbus

Attraverso la funzione registers_to_json() ho acquisito i dati dai registri Modbus. Per ogni registro definito nella struttura, il codice:

Crea l'oggetto cJSON ed inizializza il buffer dei dati Modbus a 0

Invia una richiesta Modbus al dispositivo slave utilizzando l'API mbc_master_send_request() di ESP-IDF, specificando l'indirizzo iniziale del registro, la dimensione e il comando richiesto.

Riceve una risposta contenente i dati raw provenienti dal dispositivo.

Converte i dati nei formati numerici appropriati:

Per i registri a **2 word (32 bit)**, combina i dati in un formato IEEE 754 a 32 bit e applica la scala.

Per i registri a **4 word (64 bit)**, combina i dati in un double a 64 bit.

Per i registri a **1 word (16 bit)**, moltiplica direttamente il valore grezzo per il fattore di scala.

Le conversioni utilizzano due funzioni ausiliarie:

modbus_16bit_register_to_float() per valori a 32 bit, rappresentati in formato a virgola mobile.

modbus_16bit_register_to_double() per i dati a 64 bit.

Una volta elaborati, i valori vengono organizzati come oggetti JSON, includendo sia il valore numerico sia l'unità di misura corrispondente e si ripete tutto il processo con un ritardo di 500 ms fino a che non si sono ottenuti i dati da tutti i registri scelti.

Salvataggio e debug

Il contenuto JSON viene poi scritto su un file all'interno della memoria SPIFFS tramite fopen() e fwrite(). Questo consente di mantenere i dati acquisiti, rendendoli disponibili per ulteriori analisi. La funzione di debug read_json_file() consente di leggere il contenuto del file salvato e visualizzarlo sulla console.

Ciclo main

Nel main, vengono chiamate le funzioni per inizializzare il file system e il Modbus Master. Una volta completata la configurazione, il programma entra in un ciclo infinito in cui esegue la lettura dei registri, salva i dati acquisiti in formato JSON, introduce un ritardo di 5 secondi.

4. Codice

Link al mio profilo github col codice sviluppato:

https://github.com/ciaoMondo0/Lettura-analizzatore-UPM209.git